

Environment configuration

COMPLETE LEARNING GUIDE

What You'll Learn

The exact same codebase needs to behave differently across a developer's laptop, a staging server, and production — different database URLs, different API keys, different feature flags — and configuration is the mechanism that makes this possible without maintaining separate code branches per environment. This guide covers why configuration must never be hardcoded, how `.env` files and `process.env` work in practice, structuring configuration per environment, secrets management as a distinct and higher-stakes concern than ordinary config, the Twelve-Factor App's config principle that most modern deployment platforms are built around, validating configuration at startup so misconfiguration fails loudly instead of silently, and feature flags as a form of runtime configuration.

1. Why Configuration Shouldn't Be Hardcoded

Hardcoding a database URL, API key, or environment-specific behavior directly into source code means changing environments requires changing and redeploying code — and worse, it means the exact same binary/build artifact can't be promoted unchanged from staging to production, which undermines confidence that what you tested is what you're shipping. Externalizing configuration means the application's behavior for a given environment is determined entirely by what's injected at startup, and the same tested build can be deployed anywhere by changing only its configuration.

2. `.env` Files and `process.env`

A `.env` file holds key-value pairs of environment variables for local development, loaded into `process.env` at startup by a library like `dotenv`, letting developers configure their local environment without touching code or shell profiles. In deployed environments (staging, production), these same variables are typically injected directly by the hosting platform or CI/CD system rather than from a committed file — the `.env` file itself should never be committed to version control, since it often holds real secrets for at least one environment.

BASH

```
# .env (local development only – never committed)
DATABASE_URL=postgresql://localhost:5432/lbf_dev
JWT_SECRET=dev-only-secret-not-for-prod
STRIPE_API_KEY=sk_test_xxx
```

3. Environment-Specific Configuration

Development, staging, and production typically need different values for the same configuration keys — a local database URL versus a managed production database, verbose logging in development versus structured, lower-volume logging in production, test API keys for a payment provider versus live keys. The configuration keys themselves (the schema) should stay identical across environments; only the values differ, which is exactly what lets the same code run correctly everywhere by changing only what's injected, not the code that reads it.

4. Secrets Management

Secrets (API keys, database passwords, signing keys) are configuration, but they deserve stricter handling than ordinary config like a log level or a feature flag: they should never appear in source control, never be logged, and ideally live in a dedicated secrets manager (AWS Secrets Manager, HashiCorp Vault, or a cloud platform's built-in secrets store) rather than plain environment variables in less mature setups, since a secrets manager adds audit logging, rotation support, and fine-grained access control that plain env vars don't provide.

5. The Twelve-Factor App's Config Principle

The Twelve-Factor App methodology (a widely-referenced set of best practices for building deployable, scalable web apps) states config should be stored in the environment, strictly separate from code — and specifically warns against grouping config into named 'environments' like 'development' and 'production' inside the codebase itself, since that scales poorly as more environments (a second staging environment, a per-developer preview environment) get added. The cleaner model treats each deploy as parameterized purely by whatever environment variables are injected into it at that moment, with no environment-specific code branches at all.

6. Validating Configuration at Startup

An application that silently starts up with a missing or malformed environment variable and only fails later — often confusingly, deep inside unrelated code the first time that config value is actually used — is far harder to debug than one that validates all required configuration immediately at startup and refuses to start with a clear error if anything is missing or invalid. A schema validation library (Zod, in a Node/TypeScript app) applied to `process.env` at boot time turns 'mysterious crash three requests in' into 'clear error message the moment the process starts.'

TYPESCRIPT

```
import { z } from "zod";

const envSchema = z.object({
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  PORT: z.coerce.number().default(3000),
});

export const env = envSchema.parse(process.env); // throws immediately if
invalid/missing
```

7. Feature Flags as Configuration

A feature flag is a runtime configuration value that toggles a piece of functionality on or off without a code deploy — enabling gradual rollouts, A/B tests, or an instant kill switch for a problematic new feature. Simple flags can live as ordinary environment variables/booleans; more sophisticated needs (per-user targeting, percentage rollouts, changing a flag without redeploying at all) typically call for a dedicated feature-flag service, since that goes beyond what static environment variables alone can express.

Common Pitfalls

- Committing a .env file containing real secrets to version control instead of adding it to .gitignore from the start.
- Hardcoding an API URL, key, or environment-specific value directly in source code instead of reading it from configuration.
- Using the exact same secrets (database credentials, API keys) across development, staging, and production instead of separate credentials per environment.
- Letting the application start successfully with missing or invalid configuration, only to fail confusingly later when that value is first used.
- Logging environment variables or request payloads that contain secrets, leaking them into log aggregation systems.
- Not maintaining a .env.example file documenting which environment variables are required, leaving new developers to guess or dig through code.

Key Takeaways

- Configuration must be externalized from code so the same tested build can run correctly across every environment.
- .env files are for local development convenience only and should never be committed to version control.
- Secrets deserve stricter handling than ordinary config — ideally a dedicated secrets manager with rotation and audit logging.

- Twelve-Factor config treats each deploy as parameterized by environment variables, not by environment-specific code branches.
- Validating all required configuration at startup turns silent misconfiguration into an immediate, clear failure.
- Feature flags are a form of runtime configuration that decouples deploying code from releasing functionality.

Learn More

The Twelve-Factor App — Config

<https://12factor.net/config>

dotenv — npm package documentation

<https://www.npmjs.com/package/dotenv>

OWASP — Secrets Management Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html